ALPE FORMACIÓN · SANTANDER

Edición 2026 · PHP 8.x

CURSO DE PHP · CLASE 35

API REST con Laravel

- Las rutas de la API y las versiones
- API Resources: decidir qué se publica
- El controlador de la API
- Paginación y filtros
- Autenticación con tokens: Sanctum
- Permisos en la API
- Los errores en JSON
- Probar la API
- Documentar la API



Ramón Abramo

+25 años de experiencia docente · Santander, Cantabria

ALPE

FORMACIÓN PROFESIONAL

🧭 Contenido de la clase

En la clase 26 terminamos nuestro MVC con una API REST de cursos: respuestas en JSON, verbos HTTP, códigos de estado, un token `Bearer` y errores en JSON. Nos costó lo suyo: `JsonSerializable`, `php://input`, un middleware para el token, un manejador de errores...

Hoy hacemos **la misma API en Laravel**, pero mejor: con versiones, paginación, filtros, un token distinto para cada usuario y dispositivo, y los permisos de la clase 33. Y vais a ver que casi todo el trabajo ya lo teníamos hecho: los modelos, las *policies* y hasta el *Form Request* de la web sirven tal cual.

•
👌 Cómo vamos a trabajar



1
📘 Las rutas de la API y las versiones



2
😮 API Resources: decidir qué se publica



3
🚀 El controlador de la API



4
👍 Paginación y filtros



5
📘 Autenticación con tokens: Sanctum



6
🤔 Permisos en la API



7
⚠️ Los errores en JSON



8
👌 Probar la API



9
🔍 Documentar la API



10
🚀 Ejercicios



11
👋 Lo que tenéis que llevaros de hoy





CLASE 35

👌 Cómo vamos a trabajar

Sanctum y su tabla `personal_access_tokens` ya los instalamos en la clase 28 con `php artisan install:api`. Copiad el contenido de `ejemplos/` encima del proyecto:

Esquema

```
app/Http/Resources/CursoResource.php y CategoriaResource.php ← NUEVOS
app/Http/Controllers/Api/V1/CursoController.php  ← NUEVO: el CRUD de la API
app/Http/Controllers/Api/V1/TokenController.php  ← NUEVO: pedir y anular tokens
app/Models/User.php                              ← con el trait HasApiTokens
bootstrap/app.php                                ← errores de la API en JSON y en español
routes/api.php                                   ← la versión 1 de la API
docs/API.md                                      ← NUEVO: la documentación de la API
```



💡 Para probar con comodidad, instalad en VS Code la extensión **Thunder Client**, o usad **Postman** o **Bruno**. También os pongo cada ejemplo con `curl`, que viene en Windows 10 y 11.



CLASE 35 · PUNTO 1

📘 Las rutas de la API y las versiones

Recordad de la clase 28 que las rutas de `routes/api.php` empiezan por `/api` y no usan sesión ni CSRF. Hoy las organizamos así (`routes/api.php`):

ejemplo.php — php-paso-a-paso/clase35 — Visual Studio Code

ejemplo.php

```php
// Versión 1 de la API: /api/v1/...
Route::prefix('v1')->name('api.v1.')->middleware('throttle:60,1')->group(function () {
    // Públicas
    Route::post('/tokens', [TokenController::class, 'store'])->middleware('throttle:5,1')
->name('tokens.store');
    Route::apiResource('cursos', CursoController::class)->only(['index', 'show']);

    // Con token: cabecera  Authorization: Bearer 1|abc...
    Route::middleware('auth:sanctum')->group(function () {
        Route::get('/yo', fn (Request $request) => $request->user()->only('id', 'name',
'email', 'rol'))->name('yo');
        Route::delete('/tokens/actual', [TokenController::class, 'destroy'])
->name('tokens.destroy');
        Route::apiResource('cursos', CursoController::class)->except(['index', 'show']);
    });
});
```

¿Por qué /v1?

Una web la cambiamos cuando queremos: el navegador siempre descarga la última versión. Pero una API la usan **otros programas** (una app móvil que la gente no actualiza, la web de un cliente...). Si mañana cambiamos `titulo` por `nombre`, todos esos programas se rompen.

La solución es **versionar**: la `v1` no se toca nunca. Si necesitamos cambios incompatibles, creamos una `v2` (con sus controladores en `Api\V2`) y mantenemos las dos un tiempo. Por eso el controlador vive en `app/Http/Controllers/Api/V1`.

apiResource

`Route::apiResource()` es como `Route::resource()` (clase 28), pero **sin** `create` ni `edit`: una API no muestra formularios.

Verbo	Dirección	Método
GET	`/api/v1/cursos`	`index`
POST	`/api/v1/cursos`	`store`
GET	`/api/v1/cursos/{curso}`	`show`
PUT/PATCH	`/api/v1/cursos/{curso}`	`update`
DELETE	`/api/v1/cursos/{curso}`	`destroy`

Y el `throttle:60,1` limita a 60 peticiones por minuto: una API pública sin límite es una invitación a que alguien la sature.



CLASE 35 · PUNTO 2

😮 API Resources: decidir qué se publica

En la clase 26, el modelo implementaba `JsonSerializable` para decidir qué datos salían en el JSON. En Laravel, esa tarea la hace una clase aparte: un ***API Resource*** [1].

terminal — php-paso-a-paso/clase35 — Visual Studio Code

terminal

```
php artisan make:resource CursoResource
```

`CursoResource.php`:

ejemplo.php — php-paso-a-paso/clase35 — Visual Studio Code

ejemplo.php

```php
// Decide qué datos de un curso se publican en la API y con qué forma
class CursoResource extends JsonResource
{
    public function toArray(Request $request): array
    {
        return [
            'id' => $this->id,
            'titulo' => $this->titulo,
            'slug' => $this->slug,
            'descripcion' => $this->descripcion,
            'horas' => $this->horas,
            'precio' => (float) $this->precio,
            'precio_con_iva' => $this->precio_con_iva,
            'plazas' => $this->plazas,
            'fecha_inicio' => $this->fecha_inicio?->toDateString(),
            'imagen' => $this->imagen ? asset('storage/' . $this->imagen) : null,
            // Solo si se cargó con with('categoria') / withCount('alumnos'): evita el N+1
            'categoria' => new CategoriaResource($this->whenLoaded('categoria')),
            'alumnos' => $this->whenCounted('alumnos'),
            // Solo los usuarios identificados ven si está publicado
            'publicado' => $this->when($request->user() !== null, $this->publicado),
            'enlace' => route('cursos.show', $this->resource),
        ];
    }
}
```

¿Por qué no devolver el modelo tal cual (`return $curso;`), que también sale en JSON? Por tres motivos:

1. **Seguridad.** El modelo publicaría **todas** las columnas. Con un `User`, el `remember_token` o columnas que mañana añadamos. Con un *Resource*, solo sale lo que escribimos.

2. **Estabilidad.** Si renombramos una columna en la base de datos, el *Resource* mantiene el nombre de siempre en la API y no rompemos a nadie.
3. **Formato.** Precios como número, fechas en formato estándar, la imagen con su dirección completa...

Método	Incluye el dato...
`$this->whenLoaded('categoria')`	Solo si la relación se cargó con `with()` o `load()`
`$this->whenCounted('alumnos')`	Solo si se contó con `withCount()`
`$this->when($condicion, $valor)`	Solo si se cumple la condición
`$this->whenPivotLoaded('tabla', ...)`	Solo si viene de una relación muchos a muchos

Un *Resource* puede usar otros (la categoría usa `CategoriaResource`), y el resultado sale siempre envuelto en `data`:

datos.json — php-paso-a-paso/clase35 — Visual Studio Code

datos.json

```json
{
    "data": {
        "id": 4,
        "titulo": "Diseño web con Figma",
        "precio": 180,
        "precio_con_iva": 217.8,
        "categoria": { "id": 2, "nombre": "Diseño", "slug": "diseno" },
        "alumnos": 3,
        "enlace": "http://academia-laravel.test/cursos/4"
    }
}
```



`Api/V1/CursoController.php`:

ejemplo.php — php-paso-a-paso/clase35 — Visual Studio Code

ejemplo.php

```php
// GET /api/v1/cursos/{curso}
public function show(Curso $curso): CursoResource
{
    return new CursoResource($curso->load('categoria')->loadCount('alumnos'));
}

// POST /api/v1/cursos (con token): el mismo CursoRequest que la web
public function store(CursoRequest $request): CursoResource
{
    $curso = Curso::create($request->validated());

    // Un recurso recién creado responde con 201 Created
    return new CursoResource($curso->load('categoria'));
}

// PUT /api/v1/cursos/{curso} (con token)
public function update(CursoRequest $request, Curso $curso): CursoResource
{
    $curso->update($request->validated());

    return new CursoResource($curso->load('categoria'));
}

// DELETE /api/v1/cursos/{curso} (con token): 204 No Content
public function destroy(Curso $curso): Response
{
    Gate::authorize('delete', $curso);

    $curso->delete();

    return response()->noContent();
}
```

¿Os habéis fijado? `CursoRequest` **es el mismo de la web** (clase 32). Las reglas, los mensajes en español y la autorización con la *policy*: todo reutilizado. Y cuando la petición es de la API, Laravel no redirige al formulario: responde con un **422** y los errores en JSON.

Los códigos de estado los pone Laravel solo:

Situación	Código
Devolver un *Resource*	200 OK
Devolver un *Resource* de un modelo recién creado	201 Created
`response()->noContent()`	204 No Content
`Resource::collection(...)`	200 OK



⚠️ `PUT` significa «sustituir el curso entero». Si en un `PUT` no enviáis `publicado`, `CursoRequest` lo convierte en `false` (clase 32) y el curso se despublica. Enviad siempre todos los campos.



CLASE 35 · PUNTO 4

👍 Paginación y filtros

Una API nunca debe devolver «todos» los registros: con 10.000 cursos, la respuesta sería enorme. El listado pagina y acepta filtros por la URL:

ejemplo.php — php-paso-a-paso/clase35 — Visual Studio Code

ejemplo.php

```php
// GET /api/v1/cursos?buscar=php&categoria=diseno&por_pagina=5
public function index(Request $request): AnonymousResourceCollection
{
    $porPagina = min((int) $request->query('por_pagina', 10), 50);

    $cursos = Curso::publicados()
        ->with('categoria')
        ->withCount('alumnos')
        ->when($request->query('buscar'), fn (Builder $consulta, string $texto) =>
$consulta->buscar($texto))
        ->when($request->query('categoria'), function (Builder $consulta, string $slug) {
            $consulta->whereRelation('categoria', 'slug', $slug);
        })
        ->orderBy('titulo')
        ->paginate($porPagina)
        ->withQueryString();

    // Con un paginador, la colección añade "links" y "meta" con los datos de la paginación
    return CursoResource::collection($cursos);
}
```

- `min(..., 50)`: el cliente elige cuántos quiere por página, pero **nosotros** ponemos el máximo.
- `whereRelation('categoria', 'slug', $slug)`: filtra por una columna de la relación, sin escribir el `join`.
- El segundo parámetro de la función de `when()` es el **valor** que se ha comprobado.

La respuesta trae, además de `data`, toda la información para que el cliente navegue por las páginas:

datos.json — php-paso-a-paso/clase35 — Visual Studio Code

datos.json

```
{
    "data": [ { "id": 4, "titulo": "Diseño web con Figma", ... }, ... ],
    "links": {
        "first": "http://academia-laravel.test/api/v1/cursos?categoria=diseno&page=1",
        "last": "http://academia-laravel.test/api/v1/cursos?categoria=diseno&page=3",
        "prev": null,
        "next": "http://academia-laravel.test/api/v1/cursos?categoria=diseno&page=2"
    },
    "meta": { "current_page": 1, "last_page": 3, "per_page": 2, "total": 5, ... }
}
```



CLASE 35 · PUNTO 5

📘 Autenticación con tokens: Sanctum

En la clase 26 teníamos **un** token, escrito en el `.env` y compartido por todos. **Sanctum** mejora eso: cada usuario puede pedir **sus** tokens (uno para el móvil, otro para el ordenador…) y anularlos cuando quiera [2].

Primero, el modelo `User` necesita el *trait* `HasApiTokens`:

ejemplo.php — php-paso-a-paso/clase35 — Visual Studio Code

ejemplo.php

```
use Laravel\Sanctum\HasApiTokens;

class User extends Authenticatable
{
    use HasApiTokens, HasFactory, Notifiable;
```

Y un controlador para pedir y anular tokens (`TokenController.php`):

ejemplo.php — php-paso-a-paso/clase35 — Visual Studio Code

ejemplo.php

```php
// POST /api/v1/tokens: correo + contraseña → un token
public function store(Request $request): JsonResponse
{
    $datos = $request->validate([
        'email' => ['required', 'email'],
        'password' => ['required', 'string'],
        'dispositivo' => ['required', 'string', 'max:60'],
    ]);

    $usuario = User::where('email', $datos['email'])->first();

    if ($usuario === null || ! Hash::check($datos['password'], $usuario->password)) {
        throw ValidationException::withMessages([
            'email' => 'El correo o la contraseña no son correctos.',
        ]);
    }

    // Crea el token, lo guarda cifrado en personal_access_tokens y nos da el texto UNA sola vez
    $token = $usuario->createToken($datos['dispositivo']);

    return response()->json([
        'token' => $token->plainTextToken,
        'tipo' => 'Bearer',
        'usuario' => $usuario->only('id', 'name', 'email'),
    ], 201);
}

// DELETE /api/v1/tokens/actual: anula el token con el que se hace la petición
public function destroy(Request $request): JsonResponse
{
    $request->user()->currentAccessToken()->delete();

    return response()->json(['mensaje' => 'Token anulado.']);
}
```

Esquema

```
Cliente                                        API
———————                                        ———
POST /api/v1/tokens  {email, password}   ⟶   comprueba la contraseña
                                         ⟵   201 {"token": "1|I0hg8dCg..."}
(guarda el token)

POST /api/v1/cursos
Authorization: Bearer 1|I0hg8dCg...      ⟶   auth:sanctum busca el token → es Marta
                                         ⟵   201 {"data": {...}}
```

- En la tabla `personal_access_tokens` el token se guarda **cifrado** (como una contraseña). Por eso solo se puede ver **una vez**, al crearlo.

- El middleware `auth:sanctum` lee la cabecera `Authorization: Bearer ...`, busca el token y deja al usuario en `$request->user()`. Es nuestro `TokenApi` de la clase 26.
- Como no hay sesión ni cookies, **no hace falta CSRF**: el token no lo envía el navegador solo, lo pone el programa a propósito.



💡 Sanctum tiene otro modo, para cuando el *frontend* es vuestra propia web en JavaScript (una SPA en el mismo dominio): usa la cookie de sesión normal en lugar de tokens. Los tokens son para apps móviles y programas de terceros.



💡 Los tokens pueden tener **habilidades** (*abilities*): `createToken('lector', ['cursos:leer'])` y después `$request->user()->tokenCan('cursos:leer')`. Así un token puede servir para leer pero no para escribir.



CLASE 35 · PUNTO 6

🤔 Permisos en la API

Aquí viene el regalo de haber hecho bien la clase 33: **la `CursoPolicy` funciona igual en la API**. Con el token de Marta (editora) se puede crear y modificar, pero no borrar; con el de Ramón (administrador), todo; con el de Pablo, nada.

- `store` y `update` usan `CursoRequest`, cuyo `authorize()` pregunta a la *policy*.
- `destroy` lo comprueba con `Gate::authorize('delete', $curso)`.

Y si no tiene permiso, la respuesta es un **403** en JSON. Las reglas de negocio están en **un solo sitio** y sirven para la web y para la API.



CLASE 35 · PUNTO 7

⚠️ Los errores en JSON

Gracias al `shouldRenderJsonWhen()` que ya traía `bootstrap/app.php`, los errores de `/api/...` salen en JSON. Pero salen en inglés y, con `APP_DEBUG=true`, con toda la traza del error. Los personalizamos con `render()`:

ejemplo.php — php-paso-a-paso/clase35 — Visual Studio Code

ejemplo.php

```php
->withExceptions(function (Exceptions $exceptions): void {
    $exceptions->shouldRenderJsonWhen(
        fn (Request $request) => $request->is('api/*') || $request->expectsJson(),
    );

    // Los errores HTTP de la API (403, 404, 405, 429...), con mensajes en español
    $exceptions->render(function (HttpException $e, Request $request) {
        if (! $request->is('api/*')) {
            return null;  // en la web, las páginas de error de siempre
        }

        $mensaje = match ($e->getStatusCode()) {
            403 => 'No tienes permiso para hacer esto.',
            404 => 'Recurso no encontrado.',
            405 => 'Método no permitido en esta dirección.',
            429 => 'Demasiadas peticiones. Espera un poco.',
            default => $e->getMessage(),
        };

        return response()->json(['mensaje' => $mensaje], $e->getStatusCode(), $e
->getHeaders());
    });

    $exceptions->render(function (AuthenticationException $e, Request $request) {
        if ($request->is('api/*')) {
            return response()->json(['mensaje' => 'Necesitas un token válido.'], 401);
        }
    });
})
```

Si `render()` devuelve `null`, Laravel sigue con su comportamiento normal. Es nuestro `ManejadorErrores` de la clase 26, pero solo con lo que queremos cambiar.

Código	Cuándo	Ejemplo de respuesta
401	Sin token o token anulado	`{"mensaje": "Necesitas un token válido."}`
403	La *policy* dice que no	`{"mensaje": "No tienes permiso para hacer esto."}`
404	El curso no existe (*route model binding*)	`{"mensaje": "Recurso no encontrado."}`
422	Falla la validación	`{"message": "...", "errors": {"titulo": ["..."]}}`
429	Se pasa del `throttle`	`{"mensaje": "Demasiadas peticiones. Espera un poco."}`



CLASE 35 · PUNTO 8

👌 Probar la API

Pedimos un token (en Windows, usad `curl.exe` en PowerShell, o la terminal de Laragon):

terminal — php-paso-a-paso/clase35 — Visual Studio Code

terminal

```
curl -X POST http://academia-laravel.test/api/v1/tokens -H "Accept: application/json" -H "Content-Type: application/json" -d "{\"email\":\"marta@academia-alpe.test\",\"password\":\"password\",\"dispositivo\":\"curl\"}"
```

datos.json — php-paso-a-paso/clase35 — Visual Studio Code

datos.json

```
{"token":"1|I0hg8dCgMHJk9rEX9mTkdT3urnRNPhZRxuOoAD5p0f22a69f","tipo":"Bearer","usuario":{"id":2,"name":"Marta Editora","email":"marta@academia-alpe.test"}}
```

Y lo usamos para crear un curso:

terminal — php-paso-a-paso/clase35 — Visual Studio Code

terminal

```
curl -X POST http://academia-laravel.test/api/v1/cursos -H "Accept: application/json" -H "Content-Type: application/json" -H "Authorization: Bearer 1|I0hg8dCgMHJk9rEX9mTkdT3urnRNPhZRxuOoAD5p0f22a69f" -d "{\"titulo\":\"Vue.js desde cero\",\"categoria_id\":1,\"horas\":40,\"precio\":200,\"plazas\":15,\"publicado\":true}"
```

Probad también lo que **no** debe funcionar: sin token (401), con el token de Pablo (403), con datos mal (422), un curso que no existe (404). Una API está bien hecha cuando **sus errores** son tan claros como sus aciertos.



⚠️ Enviad siempre la cabecera `Accept: application/json`. Sin ella, ante un error de validación, Laravel creería que sois un navegador e intentaría redirigiros.



CLASE 35 · PUNTO 9

🔍 Documentar la API

Una API sin documentación no la usa nadie. Como mínimo, hay que explicar: la dirección base, cómo autenticarse, cada ruta con sus parámetros y sus respuestas, y los errores posibles. Os dejo un ejemplo en

`docs/API.md`, un fichero Markdown que vive con el código y se actualiza a la vez que él.

Para proyectos grandes hay herramientas que generan la documentación **leyendo el código** (las rutas, los *Form Requests* y los *Resources*) en el formato estándar **OpenAPI**, con una web interactiva para probar cada ruta:

Herramienta	Cómo funciona
Scramble (`dedoc/scramble`)	Lee el código sin necesidad de anotaciones y publica la documentación en `/docs/api`
Scribe (`knuckleswtf/scribe`)	Genera una web estática y colecciones de Postman a partir del código y de comentarios



CLASE 35 · PUNTO 10

🚀 Ejercicios

Las soluciones se copian dentro del proyecto, respetando sus carpetas.

- ☐ **01 · Las categorías en la API.** Cread `Api\V1\CategoriaController` con `index` (cada categoría con su número de cursos) y `show` (con sus cursos publicados), y ampliad `CategoriaResource` con `whenCounted('cursos')` y `whenLoaded('cursos')`. → `CategoriaController.php`, `CategoriaResource.php` y `routes/api.php`
- ☐ **02 · Los alumnos de un curso.** Cread la ruta protegida `GET /api/v1/cursos/{curso}/alumnos`, solo para quien pueda editar el curso, con un `AlumnoResource` que incluya la fecha de matrícula y la nota con `whenPivotLoaded()`. → `AlumnoResource.php`, `CursoController.php` y `routes/api.php`
- ☐ **03 · Ordenar.** Añadid al listado el parámetro `?orden=` con las columnas `titulo`, `precio`, `horas` y `fecha_inicio` (y un `-` delante para el orden descendente: `?orden=-precio`). Cualquier otro valor se ignora: **nunca** metáis en una consulta una columna que venga de la URL sin comprobarla. → `CursoController.php`



🏆 **Reto:** cread una página `public/catalogo.html` con JavaScript que use `fetch()` para pedir `/api/v1/cursos`, pinte las tarjetas y tenga botones «Anterior» y «Siguiente» que usen los `links` de la paginación. Acabáis de separar el *frontend* del *backend*.



CLASE 35 · PUNTO 11

👋 Lo que tenéis que llevaros de hoy

Idea	En una frase
`/api/v1`	Versionar para no romper a quien ya usa la API
`apiResource()`	Las 5 rutas REST, sin formularios
API Resource	Decide qué datos salen y con qué forma; nunca el modelo tal cual
`whenLoaded`, `whenCounted`, `when`	Datos opcionales sin provocar consultas de más
`Resource::collection($paginador)`	Listado con `data`, `links` y `meta`
Sanctum	Tokens por usuario y dispositivo: `createToken()` y `auth:sanctum`
Reutilizar	El mismo *Form Request* y la misma *policy* para la web y la API
Códigos	200, 201, 204 si va bien; 401, 403, 404, 422, 429 si no
`withExceptions()` + `render()`	Errores de la API en JSON y en español
`Accept: application/json`	La cabecera que nunca debe faltar al llamar a una API
Documentación	Un `docs/API.md`, o Scramble y Scribe para generarla

📚 Fuentes y documentación oficial

1. Laravel: Eloquent API Resources
2. Laravel: Sanctum
3. Laravel: Routing, Rate Limiting
4. Laravel: Error Handling
5. MDN Web Docs: códigos de estado de respuesta HTTP
6. Scramble: documentación